

Skill1: Unified Evolution of Skill-Augmented Agents via Reinforcement Learning

Yaurui Shi^{1,2,*}, Yuxin Chen^{2,3,*}, Zhengxi Lu^{2,4}, Yuchun Miao^{2,5}, Shugui Liu¹,
Qi Gu^{2,†}, Xunliang Cai², Xiang Wang¹, An Zhang^{1,†}

¹University of Science and Technology of China, ²Meituan,

³National University of Singapore, ⁴Zhejiang University,

⁵Wuhan University, *Equal contribution.

[†]Corresponding authors: guqi03@meituan.com, an_zhang@ustc.edu.cn

Abstract

A persistent skill library allows language model agents to reuse successful strategies across tasks. Maintaining such a library requires three coupled capabilities. The agent selects a relevant skill, utilizes it during execution, and distills new skills from experience. Existing methods optimize these capabilities in isolation or with separate reward sources, resulting in partial and conflicting evolution. We propose Skill1, a framework that trains a single policy to co-evolve skill selection, utilization, and distillation toward a shared task-outcome objective. The policy generates a query to search the skill library, re-ranks candidates to select one, solves the task conditioned on it, and distills a new skill from the trajectory. All learning derives from a single task-outcome signal. Its low-frequency trend credits selection and its high-frequency variation credits distillation. Experiments on ALFWorld and WebShop show that Skill1 outperforms prior skill-based and reinforcement learning baselines. Training dynamics confirm the co-evolution of the three capabilities, and ablations show that removing any credit signal degrades the evolution. Our code is available at <https://github.com/AlphaLab-USTC/Skill1>.

1 Introduction

Reinforcement learning (RL) (Sutton and Barto, 2018; Schulman et al., 2017; Shao et al., 2024) has become an important paradigm for training large language model (LLM) agents that interact with complex environments (Guo et al., 2025; Yang et al., 2024; Team et al., 2026; Touvron et al., 2023; Shridhar et al., 2021; Yao et al., 2022a; Xi et al., 2025). Standard RL training treats each task as an isolated episode, where the strategies that lead to success are absorbed only implicitly into the policy parameters and cannot be explicitly reused on future tasks. A natural solution is to augment agents with a persistent skill library that accumulates reusable strategies from past experience, so that the agent can draw on previously successful approaches instead of solving every task from scratch (Wang et al., 2023; Zhao et al., 2024; Xia et al., 2026; Zhang et al., 2026a; Muhtar et al., 2026; Lu et al., 2026). The workflow of these skill-augmented agents can be organized around a three-stage lifecycle (Jiang et al., 2026a): skill selection, where the agent selects a relevant skill from the library for the current task; skill utilization, where the agent executes guided by the selected skill; and skill distillation, where the agent derives new reusable skills from the trajectories.

Existing methods have advanced each stage through RL, improving how agents select skills (Zhang et al., 2026b; Wang et al., 2026; Li et al., 2026a; Wu et al., 2025), utilize them (Xia et al., 2026; Muhtar et al., 2026; Zhang et al., 2026a; Li et al., 2026a; Wang et al., 2025a), and distill reusable knowledge (Zhang et al., 2026a; Wang et al., 2025a; Muhtar et al., 2026; Wu et al., 2025). Yet two fundamental questions remain open. (1) **How can an agent evolve all three capabilities**

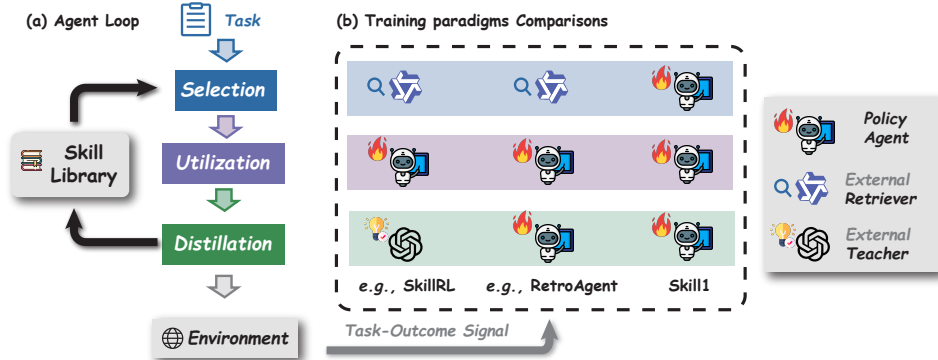


Figure 1: **Training paradigms for skill-augmented agents.** (a) The skill-augmented agent loop consists of selection, utilization, and distillation. (b) Prior methods delegate some stages to external modules without policy gradients (*e.g.*, freezes selection or uses an external teacher for distillation). Skill1 trains a single policy across all three stages with a shared task-outcome signal.

simultaneously? Existing methods apply policy updates to only a subset of the lifecycle, leaving at least one capability unoptimized, leading to optimization bottlenecks (Xia et al., 2026; Muhtar et al., 2026; Zhang et al., 2026a; Wang et al., 2025a). For example, a policy that has learned to use skills well still underperforms if it keeps routing to sub-optimal ones. **(2) How can the three capabilities co-evolve toward a shared objective?** Prior designs draw the rewards from different sources (Li et al., 2026a; Zhang et al., 2026a; Muhtar et al., 2026). For example, one capability may receive task-outcome reward while another relies on an auxiliary signal such as self-assessed quality or heuristic matching scores. Since the three capabilities jointly determine task success, optimizing them with inconsistent signals creates conflicting pressures.

We present Skill1, a framework that achieves unified evolution of skill-augmented agents by training a single policy to co-evolve skill selection, utilization, and distillation. As illustrated in Figure 1, given a new task, the policy first generates a natural-language query to retrieve candidate skills from the library, and then re-ranks the retrieved candidates to select the best match. The policy then performs multi-turn interaction with the environment conditioned on the top-ranked skill. After execution, the policy distills reusable skills from the experience based on its rollouts.

We achieve co-evolution of all three capabilities through credit assignment on a single task-outcome signal $r(\tau)$. The outcome directly measures how well the policy solves the current task and serves as the utilization reward. To credit selection and distillation, we decompose this signal into its low-frequency trend and high-frequency variation. The low-frequency trend is defined as the moving average of outcomes associated with each skill. This term reflects skill utility and guides the policy toward consistently effective skills. The high-frequency variation is approximated with the deviation of the current outcome from the trend. This term captures whether a newly distilled skill improves upon the library’s current boundary, and rewards the policy for producing useful skills.

We empirically evaluate Skill1 on ALFWorld (Shridhar et al., 2021) and WebShop (Yao et al., 2022a). Skill1 achieves 97.5% success rate on ALFWorld, surpassing all other baseline skill-augmented agents. Training dynamics confirm that selection precision, utilization success rate, and library quality improve simultaneously under the shared signal. Ablations show that removing any single stage’s credit-assignment signal degrades all three capabilities, evidencing their mutual dependence.

2 Preliminary: LLM Agent with Skill Library

Task formulation. We formulate the skill-augmented agent learning problem as a POMDP (Lauri et al., 2022) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T, \Omega, R, \gamma)$. A state $S = (x, e, \mathcal{B})$ comprises a task instruction x from dataset $\mathcal{D}$, the environment state e , and a persistent skill library $\mathcal{B} = \{s_1, s_2, \dots\}$. At each turn the agent selects an action $a \in \mathcal{A}$ to send to the environment. The observation function Ω exposes a partial view $o_t = (x, e_t, z)$, where z is the skill selected from $\mathcal{B}$ via a frozen encoder $\mathcal{E}$. The overall training objective for the workflow can be defined as:

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, \tau \sim \pi_{\theta}(\cdot|x)} [r(\tau)], \quad (1)$$

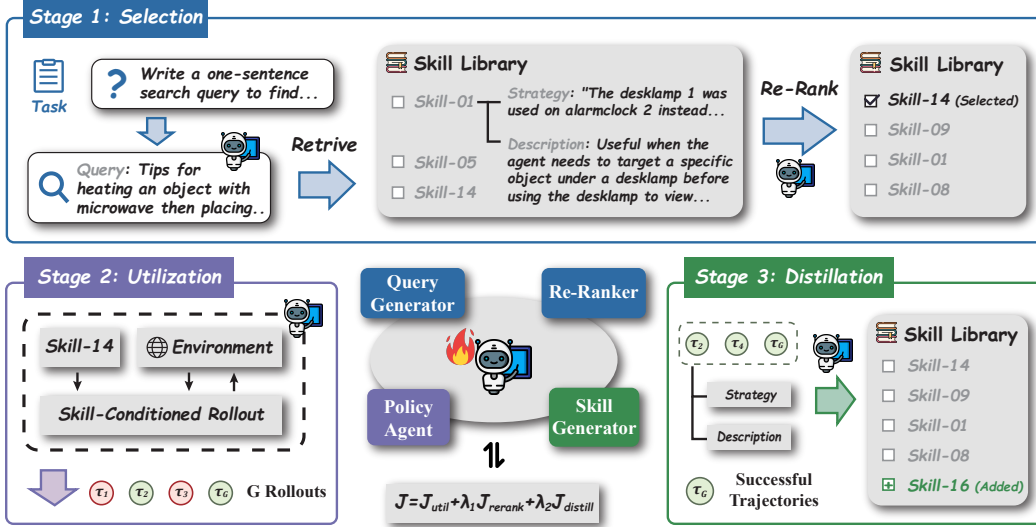


Figure 2: **Overview of the Skill1 framework.** (a) The policy generates a query and re-ranks retrieved candidates to select a skill. (b) The policy performs multi-turn interaction conditioned on the selected skill. (c) The policy reflects on the trajectory and distills a reusable skill. All learning signals are derived from the task-outcome $r(\tau)$ to achieve co-evolution of three capabilities.

where π_θ is optimized with RL algorithms such as GRPO (Shao et al., 2024) (cf. Appendix B).

Skills for LLM agents. A skill $s \in \mathcal{B}$ consists of a natural-language strategy $s.strat$ that describes how to act and a scenario description $s.desc$ that characterizes when the skill applies. The agent maintains the skill library $\mathcal{B} = \{s_1, s_2, \dots\}$ as it continuously explores the environment. To reuse a skill, the agent generates its action conditioned on the skill strategy:

$$a_t \sim \pi_\theta(\cdot \mid x, z.strat, o_{\leq t}). \quad (2)$$

To interact with a skill library, the agent selects skills from $\mathcal{B}$, utilizes them during execution (Eq. 2), and distills new skills back into $\mathcal{B}$. In §3, we show how to optimize all three stages jointly through a single policy, deriving every learning signal from the task outcome $r(\tau)$.

3 Method

We introduce Skill1, a framework that trains a single policy π_θ to co-evolve skill selection, utilization, and distillation toward a shared task-outcome objective (Figure 2). We first describe the workflow (§3.1), then derive all learning signals from the task outcome $r(\tau)$ (§3.2), and finally formulate the joint optimization objective (§3.3).

3.1 Agent Workflow

For each task $x \sim \mathcal{D}$, the policy π_θ performs three stages in sequence. A complete trajectory takes the form $\tau = (q, z, a_1, o_1, \dots, a_T, o_T, s_{new})$, where q is the selection query, z is the selected skill (or $\emptyset$), the action–observation pairs constitute the multi-turn interaction, and s_{new} is the distilled skill. The environment returns a terminal reward $r(\tau) \in \{0, 1\}$. Prompt templates are in Appendix G.

Skill selection. Given a task x , the policy generates a natural-language query $q \sim \pi_\theta(\cdot \mid x)$ to search the skill library $\mathcal{B}$. A frozen encoder $\mathcal{E}$ retrieves the top- K candidates by semantic similarity:

$$\mathcal{B}_K = \text{top-}K_{s \in \mathcal{B}} \text{ sim}(\mathcal{E}(q), \mathcal{E}(s.desc)). \quad (3)$$

The policy then re-ranks these candidates by generating a permutation $\sigma \sim \pi_\theta(\cdot \mid x, \mathcal{B}_K)$, and the top-ranked skill z is provided for utilization. Both query generation and re-ranking are produced by π_θ , so selection is directly optimizable through the policy gradient.

Skill utilization. The policy interacts with the environment for up to T turns conditioned on the selected skill: $\tau \sim \pi_\theta(\cdot \mid x, z.\text{strat}, o_{\leq t})$. For each task, G rollouts are sampled independently, each performing its own selection, utilization, and distillation.

Skill distillation. After each rollout, π_θ reflects on the trajectory to produce: (i) a reusable strategy $s_{\text{new}}.\text{strat} \sim \pi_\theta(\cdot \mid x, \tau)$ summarizing the approach, and (ii) a scenario description $s_{\text{new}}.\text{desc} \sim \pi_\theta(\cdot \mid x, \tau)$ characterizing when the skill applies. A new skill is admitted to $\mathcal{B}$ only when $r(\tau) = 1$. When the library reaches its capacity $|\mathcal{B}| = N_{\text{max}}$, the skill with the lowest retirement score $U(s) \cdot \log(n(s))$ is removed, where $n(s)$ is the number of times s has been selected. This heuristic retires skills that are both low-utility and infrequently used while preserving well-tested high-utility skills.

3.2 Reward Assignment

Co-evolution requires that each capability receives targeted learning signals from the shared task outcome $r(\tau)$. The challenge is that the three capabilities operate at different temporal scopes: utilization concerns the current episode, selection concerns which skills are consistently effective across episodes, and distillation concerns whether new experience improves upon what the library already covers. We address this by decomposing $r(\tau)$ into its low-frequency trend and high-frequency variation, assigning credit to each capability without auxiliary models or additional rollouts.

Crediting utilization. The task outcome directly measures how well the policy executes with the given skill and serves as the utilization reward:

$$R_i^{\text{util}} = r(\tau_i). \quad (4)$$

Crediting selection. Selection improves through two mechanisms. First, the query q is part of the rollout prefix and receives policy gradients through the utilization objective (Eq. 8). Better queries retrieve better candidates and lead to higher $r(\tau)$, so query quality co-improves with task performance without a dedicated reward.

Second, re-ranking requires an explicit signal that reflects long-term skill quality rather than single-episode outcomes. We maintain the trend of each skill as a per-skill utility score, updated after each rollout via exponential moving average:

$$U(s) \leftarrow (1 - \alpha) \cdot U(s) + \alpha \cdot r(\tau_i), \quad \forall s \in \mathcal{B}_K. \quad (5)$$

We update all retrieved candidates rather than only the selected one, treating co-retrieval as evidence of relevance to the same task distribution. The trend smooths out per-episode variance and accumulates each skill’s long-term contribution. We denote the best available utility as $\hat{U}_i = \max_{s \in \mathcal{B}_K^i} U(s)$, which serves as the library baseline for subsequent reward derivations. The trend supervises re-ranking by rewarding the policy for producing a permutation σ_i that agrees with the utility ordering. Here we use normalized discounted cumulative gain (NDCG) as the rubric:

$$R_i^{\text{rerank}} = \text{NDCG}(\sigma_i, \text{argsort}(-U(\mathcal{B}_K^i))). \quad (6)$$

Crediting distillation. The ideal distillation signal would measure whether a newly distilled skill improves future task performance, but that future outcome is unavailable at training time. We approximate it with the variation of the current outcome relative to the library’s trend:

$$R_i^{\text{distill}} = r(\tau_i) - \hat{U}_i, \quad (7)$$

where $\hat{U}_i = \max_{s \in \mathcal{B}_K^i} U(s)$ is the highest trend among the retrieved candidates. A positive variation indicates that the current experience surpasses what the library already covers, so the distilled skill is worth admitting. A negative variation discourages redundant distillation.

3.3 Joint Optimization

Each rollout τ_i is a concatenation of four generation segments produced by π_θ : the selection query q_i , the re-ranking permutation σ_i , the action sequence $a_{1:T}$, and the distilled skill $s_{\text{new},i}$. We assign each segment its own reward signal (§3.2) and optimize them jointly in a single gradient step using GRPO (Shao et al., 2024) (cf. Appendix B), which normalizes rewards within the G rollouts of each task into group-relative advantages.

Algorithm 1 Pseudo Code of Skill1

Require: $\pi_\theta, \mathcal{B}, \mathcal{E}, K, G, \lambda_1, \lambda_2, \alpha$

```
1: for batch of  $N$  tasks, each with  $G$  rollouts do
2:   for sample  $i = 1, \dots, N \cdot G$  do                                 $\triangleright$  Agent workflow (Sec 3.1)
3:      $q_i \leftarrow \pi_\theta(x_i)$                                            $\triangleright$  Skill selection: search
4:      $\mathcal{B}_K^i \leftarrow \text{top-}K_{s \in \mathcal{B}} \text{sim}(\mathcal{E}(q_i), \mathcal{E}(s.\text{desc}))$ 
5:      $\sigma_i \leftarrow \pi_\theta(x_i, \mathcal{B}_K^i)$ ;  $z_i \leftarrow \mathcal{B}_K^i[\sigma_i(1)]$      $\triangleright$  Skill Selection: re-rank
6:      $\tau_i \sim \pi_\theta(\cdot \mid x_i, z_i, \text{strat})$                            $\triangleright$  Skill utilization
7:      $(s_{\text{new},i}.\text{strat}, s_{\text{new},i}.\text{desc}) \leftarrow \pi_\theta(x_i, \tau_i)$      $\triangleright$  Skill distillation
8:   end for
9:    $R_i^{\text{util}} \leftarrow r(\tau_i)$ ;  $\hat{U}_i \leftarrow \max_{s \in \mathcal{B}_K^i} U(s)$          $\triangleright$  Reward assignment (Sec 3.2)
10:   $R_i^{\text{distill}} \leftarrow r(\tau_i) - \hat{U}_i$                                  $\triangleright$  Variation as distillation credit
11:   $R_i^{\text{rerank}} \leftarrow \text{NDCG}(\sigma_i, \text{argsort}(-U(\mathcal{B}_K^i)))$            $\triangleright$  Trend as selection credit
12:   $U(s) \leftarrow (1-\alpha)U(s) + \alpha r(\tau_i), \forall s \in \mathcal{B}_K^i$            $\triangleright$  Update utility scores
13:  Admit  $s_{\text{new},i}$  to  $\mathcal{B}$  if  $r(\tau_i) = 1$                            $\triangleright$  Update skill library
14:   $\theta \leftarrow \theta + \nabla_\theta [\mathcal{J}^{\text{util}} + \lambda_1 \mathcal{J}^{\text{rerank}} + \lambda_2 \mathcal{J}^{\text{distill}}]$   $\triangleright$  Joint optimization (Sec 3.3)
15: end for
```

Utilization and query. The action tokens $a_{1:T}$ are conditioned on (x_i, z_i) and optimized by the task outcome $R_i^{\text{util}} = r(\tau_i)$. The query q_i precedes the actions in the same sequence and receives gradients through the same objective:

$$\mathcal{J}^{\text{util}}(\theta) = \mathcal{J}_{\text{GRPO}}(\theta; \{\tau_1, \dots, \tau_G\}, \{\hat{A}_1, \dots, \hat{A}_G\}). \quad (8)$$

Re-ranking. The permutation σ_i is generated conditioned on the task x_i and retrieved candidates $\mathcal{B}_K^i$, and reinforced by the ranking reward R_i^{rerank} . Since different rollouts generate different queries, their retrieved candidate sets $\mathcal{B}_K^i$ differ, thus inner group comparison becomes meaningless. We thus optimize each permutation independently with a REINFORCE-style (Williams, 1992) objective:

$$\mathcal{J}^{\text{rerank}}(\theta) = \frac{1}{N \cdot G} \sum_i R_i^{\text{rerank}} \cdot \log \pi_\theta(\sigma_i \mid x_i, \mathcal{B}_K^i). \quad (9)$$

Distillation. The distilled skill tokens $(s_{\text{new},i}.\text{strat}, s_{\text{new},i}.\text{desc})$ are generated conditioned on the task x_i and trajectory τ_i , and reinforced by the variation R_i^{distill} . Advantages $\hat{A}_i^{\text{distill}}$ are normalized separately from those of utilization since the two rewards measure different aspects of same outcomes:

$$\mathcal{J}^{\text{distill}}(\theta) = \mathcal{J}_{\text{GRPO}}(\theta; \{s_{\text{new},1}, \dots, s_{\text{new},G}\}, \{\hat{A}_1^{\text{distill}}, \dots, \hat{A}_G^{\text{distill}}\}). \quad (10)$$

Total objective. All terms are combined in a single update:

$$\mathcal{J}(\theta) = \mathcal{J}^{\text{util}}(\theta) + \lambda_1 \mathcal{J}^{\text{rerank}}(\theta) + \lambda_2 \mathcal{J}^{\text{distill}}(\theta). \quad (11)$$

The utility score $U(s)$ is updated non-parametrically via Eq. (5). The full procedure is summarized in Algorithm 1. Training hyperparameter settings are in Appendix C.

4 Experiments

4.1 Experimental Setup

Environments. We evaluate on ALFWorld (Shridhar et al., 2021), a text-based household environment requiring multi-step planning and object interaction, and WebShop (Yao et al., 2022a), an online-shopping simulator where agents search and purchase products matching user specifications. We report success rate (%) on the test split for both environments.

Training. For Skill1, the initial policy is Qwen2.5-7B-Instruct (Yang et al., 2024) and the frozen encoder $\mathcal{E}$ is all-MiniLM-L6-v2 (Reimers and Gurevych, 2019). We train with GRPO under $G = 16$ and $\text{lr} = 1 \times 10^{-6}$. The skill library is initialized empty with capacity $|\mathcal{B}| \leq 5000$. The training data uses the train split of the corresponding environments. Full hyperparameters are in Appendix C.

Table 1: Main results on ALFWorld and WebShop (Success Rate, %). **Bold** denotes best results; $\uparrow$ indicates improvement over the previous best. “Avg.” stands for average success rate and “Succ.” stands for success rate.

Method	ALFWorld (Success %)							WebShop	
	Pick	Look	Clean	Heat	Cool	Pick2	Avg.	Score	Succ.
<i>w/o Training</i>									
Zero-Shot	33.4	21.6	19.3	6.9	2.8	3.2	14.8	26.4	7.8
ReAct (Yao et al., 2022b)	48.5	35.4	34.3	13.2	18.2	17.6	31.2	46.2	19.5
Reflexion (Shinn et al., 2023)	62.0	41.6	44.9	30.9	36.3	23.8	42.7	58.1	28.8
Mem0 (Chhikara et al., 2025)	54.0	55.0	26.9	36.4	20.8	7.7	33.6	23.9	2.0
ExpeL (Zhao et al., 2024)	21.0	67.0	55.0	52.0	71.0	6.0	46.3	30.9	11.2
<i>RL-Trained w/o Skills</i>									
PPO (Schulman et al., 2017)	92.3	64.0	92.5	89.5	80.3	68.8	80.4	81.4	68.7
RLOO (Ahmadian et al., 2024)	87.6	78.2	87.3	81.3	71.9	48.9	75.5	80.3	65.7
GRPO (Shao et al., 2024)	90.8	66.1	89.3	74.7	72.5	64.7	77.6	79.3	66.1
GiGPO (Feng et al., 2025)	97.7	82.7	98.8	83.7	89.3	79.2	90.8	84.4	72.8
<i>RL-Trained w/ Skills</i>									
EvolveR (Wu et al., 2025)	64.9	33.3	46.4	13.3	33.3	33.3	43.8	42.5	17.6
Mem0 (Chhikara et al., 2025) w/ GRPO	78.1	54.8	56.1	31.0	65.0	26.9	54.7	58.1	37.5
SimpleMem (Liu et al., 2026a) w/ GRPO	89.5	36.3	60.0	50.0	64.9	26.3	62.5	67.8	46.9
SkillIRL (Xia et al., 2026)	97.9	71.4	90.0	90.0	95.5	87.5	89.9	85.2	72.7
RetroAgent (Zhang et al., 2026a)	97.9	90.9	99.2	92.9	85.3	91.0	94.9	88.9	82.3
Skill1 (Ours)	100.0$\uparrow_{2.1}$	98.6$\uparrow_{7.7}$	97.3	99.2$\uparrow_{6.3}$	96.1$\uparrow_{0.6}$	96.0$\uparrow_{5.0}$	97.5$\uparrow_{2.6}$	89.7	82.9

Baselines. We compare three categories of methods in Table 1: (1) training-free agents such as ReAct (Yao et al., 2022b), Reflexion (Shinn et al., 2023), Mem0 (Chhikara et al., 2025), and ExpeL (Zhao et al., 2024); (2) RL-trained methods without skills such as PPO (Schulman et al., 2017), RLOO (Ahmadian et al., 2024), GRPO (Shao et al., 2024), and GiGPO (Feng et al., 2025); and (3) RL-trained methods with skills such as EvolveR (Wu et al., 2025), Mem0 and SimpleMem (Liu et al., 2026a) optimized with GRPO, SkillIRL (Xia et al., 2026), and RetroAgent (Zhang et al., 2026a). All baselines use the same base model Qwen2.5-7B-Instruct for fair comparison.

4.2 Main Results

Table 1 presents the main results. We reproduce RetroAgent with the official implementation and borrow other baseline results from prior research (Feng et al., 2025; Xia et al., 2026; Jiang et al., 2025a). Skill1 results are averaged across three runs, and we report statistical analysis in Appendix D.

Skill1 achieves the highest overall performance. On ALFWorld, Skill1 reaches 97.5% average success rate, surpassing the previous best RetroAgent by 2.6 points and ranking first on 5 out of 6 task types. On WebShop, Skill1 also demonstrates the best performance across all methods.

An explicit skill library complements parameter-only RL. GiGPO, the strongest RL-only method, absorbs strategies implicitly into parameters and cannot explicitly reuse them across tasks. Skill1 surpasses it by 6.5 points, with the largest gains on Look and Pick2 where composing multiple sub-procedures benefits most from reusable skills.

Unified optimization outperforms methods that leave part of the lifecycle unoptimized. RetroAgent optimizes utilization and distillation with separate intrinsic rewards but provides no gradient signal for selection. SkillIRL freezes its selection mechanism after cold-start SFT. Skill1 optimizes all three stages jointly through a single task-outcome signal. The comparison reveals a clear trend that agent performance increases with the degree of co-evolution.

4.3 Analysis

4.3.1 Ablation Study

We remove workflow components and zero out auxiliary objective weights to isolate each design choice. All variants share the same base model and training budget. Results are reported in Table 2.

The skill library is the foundation, and distillation makes it effective. Removing the library entirely causes the largest drop, from 97.5% to 80.9%, with Heat and Pick2 losing over 28 points each. These task types require composing multi-step sub-procedures that benefit most from reusable skills. Removing distillation while keeping the library still reduces performance by 5.1 points. Without

Table 2: Ablation study on ALFWorld (Success Rate %). Upper block ablates workflow components; lower block ablates training objectives.

	Pick	Look	Clean	Heat	Cool	Pick2	Avg.
Skill1	100.0	98.6	97.3	99.2	96.1	96.0	97.5
w/o Selection	96.9	90.3	98.0	90.4	86.5	85.3	91.8
w/o Distillation	97.4	88.5	98.1	96.1	87.6	89.5	92.4
w/o Library	96.7	71.5	94.9	70.7	71.5	65.5	80.9
w/ $\lambda_1=0$	99.5	80.5	98.8	100.0	90.6	84.9	94.0
w/ $\lambda_2=0$	100.0	85.4	95.5	96.4	91.0	96.2	94.9
w/ $\lambda_1=\lambda_2=0$	98.1	74.9	95.6	95.6	79.5	87.2	90.2

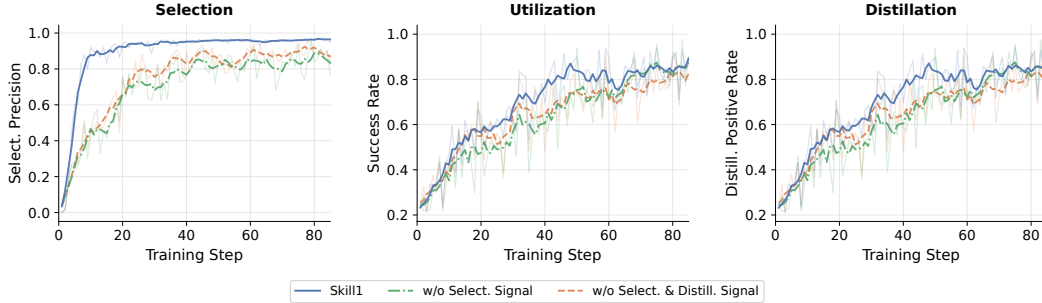


Figure 3: Training dynamics of the three capability metrics. Full Skill1 achieves fast and unified convergence across all stages. Removing selection signal (green) or both selection and distillation signals (orange) slows convergence of all capabilities.

distillation the library stores raw trajectories rather than condensed strategies, making selection noisier and reuse less effective.

Selection loss propagates to downstream stages. Without selection the average drops by 5.7 points, concentrated on Heat and Pick2 where routing to the correct multi-step skill matters most. Notably, this degradation occurs even though the utilization reward remains intact, showing that poor skill routing bottlenecks the entire pipeline regardless of the policy’s solving ability.

The two auxiliary objectives are complementary. Setting $\lambda_1=0$ or $\lambda_2=0$ individually reduces performance by 3.5 and 2.6 points respectively. Removing both yields a sharper decline to 90.2%, worse than removing each stage individually. This gap shows that the signals benefit utilization beyond their direct targets, confirming that both signals are necessary to sustain full co-evolution.

4.3.2 Co-evolution Dynamics

Figure 3 tracks three capability metrics across training: (1) selection precision, the average skill utility scores $U(s)$; (2) task-outcome reward $r(\tau)$ for utilization; and (3) distillation positive rate, the fraction of new rollouts exceeding the average of retrieved ones $\hat{U}_i$. We compare the full system against ablations that progressively remove credit-assignment signals.

The three capabilities exhibit mutual reinforcement under unified training. Selection precision converges first, reaching 0.95 by step 20. The resulting high-quality skill supply then accelerates the other two stages, with both utilization and distillation reaching 0.8 by step 60. This sequential acceleration shows that improvements in one stage propagate forward through the lifecycle.

Ablating any credit-assignment signal slows all three capabilities. Removing the selection signal reduces selection precision as expected, but also drags down utilization and distillation because the policy routes to sub-optimal skills more frequently. Further removing distillation causes utilization scores to drop, even though it still receives its own direct reward. This suggests that each signal contributes to the overall growing trend, which is a direct evidence of co-evolution.

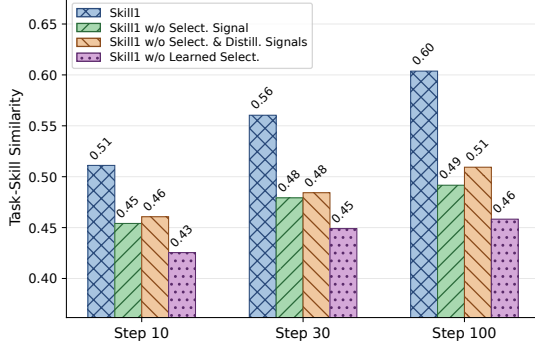


Figure 4: Task-skill similarity at three training checkpoints. The trend signal drives continuous improvement in selection quality.

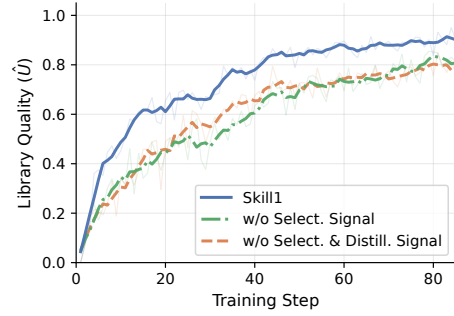


Figure 5: Top-skill utility ($\hat{U}$) during training. The variation signal drives the policy to distill increasingly effective skills.

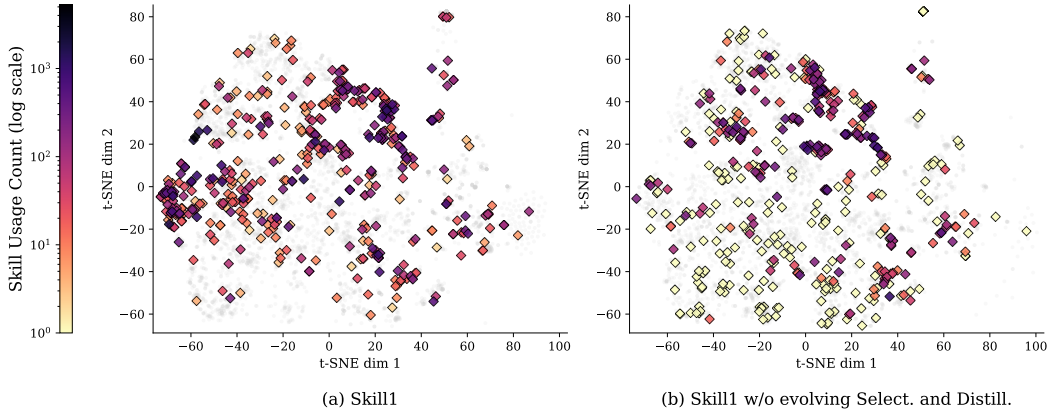


Figure 6: T-SNE visualization of the skill libraries after convergence, with and without RL-trained selection and distillation. The top-10 percent most frequently used skills are highlighted. Skill1 activates nearly twice as many high-frequency skills, and these skills span a broader strategy space.

4.3.3 Evolution of Skill Management Capabilities

The previous section shows that capability metrics rise together. Here we examine the qualitative nature of that improvement: does the policy actually learn to select more relevant skills and distill higher-quality ones?

The policy learns to generate increasingly precise selection queries. Figure 4 measures task-skill similarity at three checkpoints. Full Skill1 improves from 0.51 to 0.60 across training because the trend signal rewards queries that retrieve historically high-utility skills, gradually sharpening the policy’s ability to describe what it needs. Removing the selection signal slows this learning, and without learned selection entirely, similarity stays almost flat at the lowest level.

The library ceiling rises as the policy learns to distill better skills. Figure 5 tracks $\hat{U}$, the utility of the top-ranked skill per task. A rising $\hat{U}$ means increasingly effective skills are entering the library, not merely more skills. Full Skill1 reaches 0.91 by step 85 while both ablations lag by approximately 0.10. The variation signal creates this pressure: producing a skill similar to existing ones yields little reward, so the policy must discover genuinely better strategies to obtain positive gradient.

4.3.4 Skill Library Diversity

We examine whether the library is utilized as a diverse collective asset or collapses to a few dominant entries. Figure 6 visualizes the converged libraries with and without credit-assignment signals.

Table 3: Computational cost on ALFWorld training. We report wall-clock time per step (seconds) and library size (number of skills) at three checkpoints.

Method	Time / Step (s)			Library Size		
	Step 20	Step 60	Step 100	Step 20	Step 60	Step 100
GRPO (no library)	301.3	274.1	296.7	—	—	—
SkillRL	368.1	319.0	326.6	60	71	83
Skill1	386.6	444.3	493.8	915	3,899	5,000
w/o Select. Step	367.4	406.7	521.8	892	3,693	5,000
w/o Distill. Step	508.8	750.1	738.4	2,212	5,000	5,000

Co-evolution activates a broader set of skills. Skill1 frequently use a broader set of skills. As observed in Figure 6, the skill usage count distributes more uniformly in the left panel. Without evolving signals (*i.e.*, Skill1 w/o Select. and Distill.), the skill usage count distribution sharpens, where only a few amount of popular skills are intensively utilized.

Frequently used skills cover diverse strategies. We also observe that the active skills in Skill1 span a much broader region of the strategy space. In the contrary, the popular skills (red and purple ones) on the right subfigure huddle together with only limited coverage. In the design of our method, producing a under-performing skill similar to existing ones yields negative reward, so the policy is pressured to cover underserved scenarios rather than duplicating successful ones.

4.3.5 Computational Overhead

We compare wall-clock time and library size for Skill1, SkillRL, and two ablations under identical hardware of 8 H800 80GB GPUs.

Skill1 adds moderate overhead over baseline methods. GRPO without a library runs at approximately 290s per step. SkillRL maintains near-constant cost because its library grows minimally from 60 to 83 skills, but this static library limits final performance to 89.9% compared to 97.5% for Skill1. Skill1 operates at 387 to 494s, roughly 1.3 to 1.7 times slower than GRPO, with the increase stemming from the growing library context. The selection step itself adds negligible overhead as query generation and re-ranking operate on short sequences compared to multi-turn interactions against the environment.

Distillation controls both library quality and computational cost. Without distillation, raw trajectories enter the library directly, growing it at 2.4 times the rate of Skill1. The larger library lengthens the selection context, making the variant without distillation 69% slower by step 60 and saturating the 5,000-skill cap far earlier. Distillation compresses experience into concise skills, simultaneously improving quality and bounding cost.

5 Conclusion and Limitations

Conclusion. We present Skill1, a framework that trains a single policy to co-evolve skill selection, utilization, and distillation toward a shared task-outcome objective. By decomposing this signal into its low-frequency trend and high-frequency variation, Skill1 derives per-capability credit assignment without auxiliary rewards. Experiments on ALFWorld and WebShop show consistent gains over prior skill-based and RL baselines, and ablations confirm that the three capabilities evolve in a coupled manner. We hope this unified perspective encourages further research on jointly optimizing the full skill lifecycle in broader agent settings.

Limitations. While Skill1 achieves strong performance, several limitations remain.

- **Environment coverage.** Our evaluation is limited to two representative text-based agent environments. Whether the co-evolution framework generalizes to more environments (*e.g.*, deep search environments) or those with visual observations remains unexplored.
- **Scalability of the skill library.** The library capacity in this work is capped at 5,000 entries. As the diversity of tasks grows, the fixed-size library may become a bottleneck, and more sophisticated eviction or hierarchical organization strategies may be required.

References

- Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Meituan LongCat Team, Anchun Gui, Bei Li, Bingyang Tao, Bole Zhou, Borun Chen, Chao Zhang, Chen Gao, Chen Zhang, Chengcheng Han, et al. Longcat-flash-thinking-2601 technical report. *arXiv preprint arXiv:2601.16725*, 2026.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 20744–20757. Curran Associates, Inc., 2022a.
- Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, et al. Agentgym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 27914–27961, 2025.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Intrinsically-Motivated and Open-Ended Learning Workshop@ NeurIPS2023*, 2023.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- Xiaoying Zhang, Zichen Liu, Yipeng Zhang, Xia Hu, and Wenqi Shao. Retroagent: From solving to evolving via retrospective dual intrinsic feedback. *arXiv preprint arXiv:2603.08561*, 2026a.
- Dilxat Muhtar, Jiashun Liu, Wei Gao, Weixun Wang, Shaopan Xiong, Ju Huang, Siran Yang, Wenbo Su, Jiamang Wang, Ling Pan, et al. Complementary reinforcement learning. *arXiv preprint arXiv:2603.17621*, 2026.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026.

- Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026a.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026b.
- Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. Skillorchestra: Learning to route agents via skill transfer. *arXiv preprint arXiv:2602.19672*, 2026.
- Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning. *arXiv preprint arXiv:2603.16060*, 2026a.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- Jiong Xiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025a.
- Mikko Lauri, David Hsu, and Joni Pajarinen. Partially observable markov decision processes in robotics: A survey. *IEEE Transactions on Robotics*, 39(1):21–40, 2022.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022b.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, 2024.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026a.
- Yulun Jiang, Liangze Jiang, Damien Teney, Michael Moor, and Maria Brbic. Meta-rl induces exploration in language agents. *arXiv preprint arXiv:2512.16848*, 2025a.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

- Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. Archer: Training language model agents via hierarchical multi-turn rl. In *International Conference on Machine Learning*, pages 62178–62209. PMLR, 2024.
- Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*, 2025.
- Pranav Putta, Edmund Mills, Naman Garg, Sumeet Motwani, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent q: Advanced reasoning and learning for autonomous ai agents. *arXiv preprint arXiv:2408.07199*, 2024.
- Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and error: Exploration-based trajectory optimization of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 7584–7600, 2024.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025b.
- Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, Jie Tang, and Yuxiao Dong. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. *arXiv preprint arXiv:2510.04206*, 2025a.
- Yulun Jiang, Liangze Jiang, Damien Teney, Michael Moor, and Maria Brbic. Meta-rl induces exploration in language agents. *arXiv preprint arXiv:2512.16848*, 2025b.
- Hanlin Wang, Chak Tou Leong, Jiashuo Wang, Jian Wang, and Wenjie Li. Spa-rl: Reinforcing llm agents via stepwise progress attribution. *arXiv preprint arXiv:2505.20732*, 2025c.
- Quan Wei, Siliang Zeng, Chenliang Li, William Brown, Oana Frunza, Wei Deng, Anderson Schneider, Yuriy Nevmyvaka, Yang Katie Zhao, Alfredo Garcia, and Mingyi Hong. Reinforcing multi-turn reasoning in llm agents via turn-level reward design. *arXiv preprint arXiv:2505.11821*, 2025a.
- Jingtong Gao, Ling Pan, Yejing Wang, Rui Zhong, Chi Lu, Qingpeng Cai, Peng Jiang, and Xiangyu Zhao. Navigate the unknown: Enhancing llm reasoning with intrinsic motivation guided exploration. *arXiv preprint arXiv:2505.17621*, 2025.
- Jiawei Wang, Jiakai Liu, Yuqian Fu, Yingru Li, Xintao Wang, Yuan Lin, Yu Yue, Lin Zhang, Yang Wang, and Ke Wang. Harnessing uncertainty: Entropy-modulated policy gradients for long-horizon llm agents. *arXiv preprint arXiv:2509.09265*, 2025d.
- David Abel, André Barreto, Benjamin Van Roy, Doina Precup, Hado P van Hasselt, and Satinder Singh. A definition of continual reinforcement learning. *Advances in Neural Information Processing Systems*, 36:50377–50407, 2023.
- Runzhe Zhan, Yafu Li, Zhi Wang, Xiaoye Qu, Dongrui Liu, Jing Shao, Derek F Wong, and Yu Cheng. Exgrpo: Learning to reason from experience. *arXiv preprint arXiv:2510.02245*, 2025.
- Tianzhu Ye, Li Dong, Qingxiu Dong, Xun Wu, Shaohan Huang, and Furu Wei. Online experiential learning for language models. *arXiv preprint arXiv:2603.16856*, 2026.
- Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H Chi, et al. Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025b.
- Zeyuan Liu, Jeonghye Kim, Xufang Luo, Dongsheng Li, and Yuqing Yang. Exploratory memory-augmented llm agent via hybrid on- and off-policy optimization. In *The Fourteenth International Conference on Learning Representations*, 2026b.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025.

- Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. *arXiv preprint arXiv:2508.16153*, 2025.
- Sai Wang, Yu Wu, and Zhongwen Xu. Cogito, ergo ludo: An agent that learns to play by reasoning and planning. *arXiv preprint arXiv:2509.25052*, 2025e.
- Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2):235–256, 2002.
- Xiaoying Zhang, Yipeng Zhang, Hao Sun, Kaituo Feng, Chaochao Lu, Chao Yang, and Helen Meng. Critique-grpo: Advancing llm reasoning with natural language and numerical feedback. *arXiv preprint arXiv:2506.03106*, 2025b.
- Jonas Hübötter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- Weiran Yao, Shelby Heinecke, Juan Carlos Niebles, Zhiwei Liu, Yihao Feng, Le Xue, Rithesh Murthy, Zeyuan Chen, Jianguo Zhang, Devansh Arpit, et al. Retroformer: Retrospective large language agents with policy gradient optimization. *arXiv preprint arXiv:2308.02151*, 2023.
- Tennison Liu and Mihaela Van Der Schaar. Position: Truly self-improving agents require intrinsic metacognitive learning. In *Forty-second International Conference on Machine Learning Position Paper Track*, 2025.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026b.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, et al. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*, 2026b.
- Anthropic. Introducing agent skills. *Claude Blog*, 2025.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.
- Jingyang Qiao, Weicheng Meng, Yu Cheng, Zhihang Lin, Zhizhong Zhang, Xin Tan, Jingyu Gong, Kun Shao, and Yuan Xie. Memory intelligence agent. *arXiv preprint arXiv:2604.04503*, 2026.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.

A Related Work

Reinforcement Learning for LLM Agents. Core algorithmic advances include GRPO (Shao et al., 2024), anchor-state grouping (Feng et al., 2025), and dynamic sampling with asymmetric clipping (Yu et al., 2025). Multi-turn RL methods address long-horizon challenges through hierarchical value functions (Zhou et al., 2024), leave-one-out advantage estimation (Chen et al., 2025), MCTS-guided search (Putta et al., 2024), exploration-based trajectory optimization (Song et al., 2024), multi-turn self-evolution (Wang et al., 2025b; Zhang et al., 2025a), and cross-episode meta-RL (Jiang et al., 2025b). Recent work further refines credit assignment via stepwise progress attribution (Wang et al., 2025c; Wei et al., 2025a) or intrinsic exploration signals (Gao et al., 2025; Wang et al., 2025d). Prompt-based methods such as ReAct (Yao et al., 2022b) and Reflexion (Shinn et al., 2023) enable reasoning without parameter updates but are upper-bounded by the frozen policy (Abel et al., 2023). Skill1 extends GRPO by decomposing a single task-outcome signal into stage-specific gradients for selection, utilization, and distillation within a unified RL objective.

Experience Reusing. Structuring past experience for reuse improves RL sample efficiency (Zhan et al., 2025; Ye et al., 2026; Muhtar et al., 2026), and explicit memory systems that store interaction histories (Wei et al., 2025b; Liu et al., 2026a,b) or distilled lessons (Fang et al., 2025; Zhou et al., 2025; Wang et al., 2025e) support continuous adaptation. RetroAgent (Zhang et al., 2026a) combines intrinsic progress rewards with language-based lesson extraction and a utility-aware selection strategy (Auer et al., 2002). Critique-GRPO (Zhang et al., 2025b) integrates natural-language critiques with numerical rewards, and RL-based self-distillation (Hübottter et al., 2026) refines failed trajectories into policy updates. Retrospective self-correction through natural-language critiques (Madaan et al., 2023; Yao et al., 2023) further enables agents to learn from failures (Liu and Van Der Schaar, 2025). Skill1 builds on these insights but derives all learning signals from a single task-outcome signal, eliminating the need for separate intrinsic reward design.

Skill Libraries for LLM Agents. A growing body of work equips LLM agents with persistent skill libraries (Jiang et al., 2026a; Xu and Yan, 2026; Li et al., 2026b; Jiang et al., 2026b; Anthropic, 2025). For selection, approaches include frozen embedding selectors (Xia et al., 2026; Muhtar et al., 2026), heuristic scoring (Zhang et al., 2026a), learned routing (Zhang et al., 2026b; Wang et al., 2026), and policy log-probability ranking (Li et al., 2026a; Wu et al., 2025). For utilization, RL-based methods condition the policy on selected skills (Xia et al., 2026; Muhtar et al., 2026; Zhang et al., 2026a; Li et al., 2026a; Wang et al., 2025a), sometimes with hierarchical rewards to incentivize skill use (Li et al., 2026a; Muhtar et al., 2026). For distillation, methods range from prompt-based extraction (Zhao et al., 2024) and training-free skill versioning (Yang et al., 2026) to teacher-driven generation (Xia et al., 2026), co-evolving extractors (Muhtar et al., 2026), and self-reflection (Zhang et al., 2026a; Wang et al., 2025a; Wu et al., 2025; Qiao et al., 2026). Existing methods have not yet achieved RL-optimized status on all three stages simultaneously, and those that optimize multiple stages use heterogeneous learning signals without a unified objective. Skill0 (Lu et al., 2026) internalizes skills into model parameters with zero external skills; Skill1 co-evolves all three stages through one policy model and a unified task outcome signal.

B Algorithm Details

We use Group Relative Policy Optimization (GRPO) (Shao et al., 2024) as the optimization method, which eliminates the need for a separate value network by computing advantages relative to a group of rollouts sampled from the same task. For each task d , a group of G rollouts $\{\tau_i\}_{i=1}^G$ is sampled from $\pi_{\theta_{\text{old}}}$. The group-relative advantage for rollout i is:

$$\hat{A}_i = \frac{r(\tau_i) - \text{mean}(\{r(\tau_1), \dots, r(\tau_G)\})}{\text{std}(\{r(\tau_1), \dots, r(\tau_G)\})}. \quad (12)$$

Let $\rho_t^{(i)}(\theta) = \pi_\theta(a_t^{(i)} | s_t^{(i)}) / \pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})$ denote the per-token importance ratio. The GRPO objective maximizes the clipped surrogate:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau_i|} \sum_{t=1}^{|\tau_i|} \min(\rho_t^{(i)} \hat{A}_i, \text{clip}(\rho_t^{(i)}, 1-\epsilon, 1+\epsilon) \hat{A}_i) - \beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}], \quad (13)$$

where ϵ is the clipping ratio, β controls KL regularization toward a reference policy π_{ref} , and $|\tau_i|$ is the number of tokens in rollout i .

C Implementation Details

Training infrastructure. Skill1 is trained on 8 NVIDIA H800-80GB GPUs using the VeRL framework (Sheng et al., 2024) with Fully Sharded Data Parallelism (FSDP) under BFloat16 precision. Rollout generation uses vLLM with tensor parallelism of 4. Training converges in approximately 100 to 150 steps (roughly 30 hours on ALFWorld). The auxiliary objective weights are $\lambda_1 = \lambda_2 = 0.3$ throughout all experiments unless otherwise specified.

Baseline reproduction. We reproduce RetroAgent using its official implementation.¹ For SkillRL, EvolveR, Mem0, and SimpleMem, we use numbers reported in their respective papers (Xia et al., 2026; Wu et al., 2025; Chhikara et al., 2025; Liu et al., 2026a) under the same base model (Qwen2.5-7B-Instruct). GiGPO results are taken from Feng et al. (2025). All RL baselines use identical training budgets (150 epochs) and the same train/test splits to ensure fair comparison.

Hyperparameters. Table 4 lists the shared training hyperparameters across both environments. Table 5 lists the per-environment differences. Table 6 lists the skill library configuration.

Table 4: Shared training hyperparameters.

Hyperparameter	Value
<i>Optimization</i>	
Algorithm	GRPO
Learning rate	1×10^{-6}
KL loss coefficient	0.01
KL loss type	low-variance KL
PPO mini-batch size	256
PPO micro-batch size per GPU	16
Gradient checkpointing	True
Re-ranking loss weight λ_1	0.3
Distillation loss weight λ_2	0.3
<i>Rollout</i>	
Group size G	16
Max prompt length	16,384 tokens
Max response length	2,048 tokens
vLLM tensor parallelism	4
GPU memory utilization	0.7
Validation temperature	0.4

Table 5: Per-environment hyperparameters.

Hyperparameter	ALFWorld	WebShop
Training batch size	16	32
Validation batch size	64	128
Max environment steps	50	15

D Statistical Analysis

We run all methods with 3 independent random seeds and report mean $\pm$ standard deviation ($1-\sigma$). The primary source of variability is the random seed, which affects parameter initialization, rollout sampling order, and skill library evolution trajectory. We use SciPy’s `ttest_ind` with `equal_var=False` (Welch’s t-test) to assess statistical significance.

¹<https://github.com/zhangxy-2019/RetroAgent>

Table 6: Skill library configuration.

Parameter	Value
<i>Selection</i>	
Encoder	all-MiniLM-L6-v2 (384-dim)
Top- K candidates	5
Training selection strategy	UCB
Evaluation selection strategy	Greedy
UCB exploration scale	1.0
Similarity weight w_{sim}	0.6
<i>Library Management</i>	
Maximum library size	5,000
Utility EMA rate α	0.05

D.1 Full Performance Breakdown

We select RetroAgent as the strongest baseline and run it with 3 independent seeds under identical conditions to obtain variance estimates. Figure 7 reports per-task-type success rates (mean $\pm$ std) on ALFWorld.

D.2 Analysis

Skill1 achieves statistically significant improvement over RetroAgent. On the aggregate metric (ALF All), Skill1 achieves 97.5 ± 0.6 versus RetroAgent’s 94.9 ± 0.9 . A Welch’s t-test on the 3-seed averages yields $t = 4.06$, $df = 3.40$, $p = 0.021$ (< 0.05). The result confirms that the gain is not attributable to seed variance. Per-task significance is strongest on the tasks where RetroAgent struggles most: Heat ($p = 0.004$), Cool ($p = 0.005$), and Look ($p = 0.020$).

Skill1 exhibits lower aggregate variance than RetroAgent. Skill1’s overall standard deviation (0.6) is smaller than RetroAgent’s (0.9), indicating more stable convergence across seeds. The unified evolution framework, where selection, utilization, and distillation reinforce each other, reduces sensitivity to initialization.

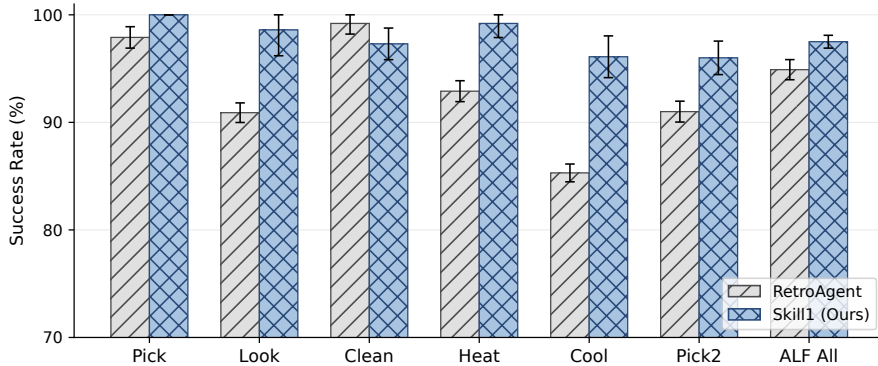


Figure 7: Per-task success rates (mean $\pm$ std over 3 seeds). Skill1 consistently outperforms RetroAgent across all task types.

E Broader Impacts

This work develops a framework for LLM agents to autonomously acquire and reuse behavioral skills through reinforcement learning. On the positive side, the approach can reduce the manual engineering effort required to build capable agents and enable more sample-efficient learning in interactive environments.

On the negative side, agents that autonomously accumulate skills may exhibit emergent behaviors that are difficult to predict or audit. In high-stakes deployment scenarios, an unconstrained skill library could encode harmful action sequences, and brings new injection risks. We recommend deploying such systems with human-in-the-loop oversight and constraining the action space in safe domains.

F Case Studies

We present two representative case studies from the ALFWorld evaluation, comparing Skill1 against RetroAgent on the same test task. Each case demonstrates a different transfer mechanism (failure avoidance and error correction) and highlights why unified evolution of selection, utilization, and distillation produces qualitatively different behavior from baselines that lack joint optimization.

Case Study 1: Failure Avoidance via Appliance Selection

Test task: heat some plate and put it in cabinet.

Challenge: The environment contains both stoveburners and a microwave. Stoveburners are non-functional for plates, but this constraint is invisible from the task description.

Skill1: Retrieved Skill (Utility: 0.951, Used 593 times)

Origin task: heat some plate and put it in fridge.

Description: Useful when the agent needs to heat an object using a microwave, especially when stove burners are not effective.

Strategy: “Successfully used microwave 1 to heat the plate. The attempt failed to heat the plate using the stoveburners, which might not have been operational. The microwave 1 was found on countertop 2.”

Skill1 reasoning (step 3): “I have already picked up the plate from countertop 1. **The microwave 1 is the most suitable option for heating the plate, as mentioned in past reflections.** I will go to the microwave 1 to heat the plate.”

Skill1 execution trace (6 actions, optimal):

1. go to countertop 1
2. take plate 1 from countertop 1
3. go to microwave 1
4. heat plate 1 with microwave 1
5. go to cabinet 1
6. put plate 1 in cabinet 1 ✓ SUCCESS

Case Study 2: Error Correction via Object Targeting

Test task: examine the alarmclock with the desk lamp.

Challenge: The room contains multiple alarmclock instances. Targeting the wrong instance leads to task failure.

Skill1: Retrieved Skill (Utility: 0.997, Used 112 times)

Origin task: look at alarmclock under the desk lamp.

Description: Useful when the agent needs to target a specific object under a desk lamp before using the desk lamp to view the object.

Strategy: “The desk lamp 1 was used on alarmclock 2 instead of the alarmclock under the desk lamp. The agent should locate the alarmclock under the desk lamp first before using the desk lamp.”

Skill1 reasoning (step 1): “I need to find an alarmclock and use the desk lamp on it. I’ll start by examining the dresser 1 to find both the alarmclock and desk lamp.”

Skill1 execution trace (4 actions, minimal):

1. go to dresser 1
2. take alarmclock 1 from dresser 1
3. use desk lamp 1 ✓ SUCCESS

Discussion. Both cases illustrate how the co-evolved skill library captures knowledge that goes beyond surface-level pattern matching. Case 1 demonstrates failure avoidance: RetroAgent lacks a high-utility skill encoding the stoveburner constraint because its selection mechanism is not optimized to route heat-tasks to the relevant skill. Skill1 retrieves the correct skill and explicitly cites it in its reasoning chain. Case 2 demonstrates error correction: RetroAgent picks the wrong alarmclock instance because its library does not preserve the targeting lesson from prior failures with sufficient utility. Skill1’s variation-driven distillation retains such lessons and the trend-driven selection surfaces them at test time. In both cases, Skill1 achieves near-optimal trajectories while the baseline exhausts steps on avoidable mistakes.

G Prompt Templates

We list the prompt templates used in each stage of Algorithm 1:

- **Selection (Query generation)** (line 4): π_θ generates query q to retrieve candidates from $\mathcal{B}$.
- **Selection (Re-ranking)** (line 6): π_θ ranks $\mathcal{B}_K$ and selects the top skill z .
- **Utilization** (line 8): π_θ interacts with the environment conditioned on $z.\text{strat}$.
- **Distillation** (line 9): π_θ reflects on τ and produces s_{new} .

G.1 ALFWorld

Query Generation

Task: {TASK}
 Observation: {INITIAL_OBSERVATION}
 Write a one-sentence search query to find relevant past experiences for this task. Do NOT output an action.
 Example: <query>tips for heating an object with microwave then placing it</query>
 <query>

Re-ranking

You are about to attempt a task in the ALFRED Embodied Environment.
 Task: {TASK}
 Initial Observation: {INITIAL_OBSERVATION}
 Below are $\{K\}$ past experiences retrieved from memory. Each is labeled with an ID.
 {CANDIDATE_EXPERIENCES}
 Rank these experiences from MOST useful to LEAST useful for the current task. Consider which experience addresses the specific challenges you expect to face.
 Output ONLY the ranked IDs as a comma-separated list within <rank> </rank> tags.

Utilization

You are an expert agent operating in the ALFRED Embodied Environment. Your task is to: {TASK}
[Injected if a skill is selected:]
 Past reflections on similar tasks: {SKILL.strat}
 Warning: These lessons may be outdated. Use them only if they align with your current observation.
 Prior to this step, you have already taken $\{N\}$ step(s). Below are the most recent $\{W\}$ observations and the corresponding actions you took: {ACTION_HISTORY}
 You are now at step {CURRENT_STEP} and your current observation is: {OBSERVATION}
 Your admissible actions of the current situation are: [{ADMISSIBLE_ACTIONS}].
 You should first reason step-by-step within <think> </think> tags. Then choose an admissible action within <action> </action> tags.

Distillation

You are an expert evaluating an ALFRED Embodied Environment task attempt.
Your task is to: {TASK}
The task was {successfully/unsuccessfully} completed.
Trajectory of the attempt: {TRAJECTORY}
<think> Analyze: What subtasks were attempted (pick up, navigate, use appliance, place)? Which succeeded or failed? What specific actions led to this outcome? What is the most valuable lesson?
</think>
Output your evaluation as JSON:
{
 "task_success": ...,
 "action_lesson": "...",
 "navigation_lesson": "...",
 "description_head": "[WHEN this lesson is useful -- general task type, not specific task]"
}

G.2 WebShop

Query Generation

Task: {TASK}
Observation: {INITIAL_OBSERVATION}
Write a one-sentence search query to find relevant past experiences for this task. Do NOT output an action.
Example: <query>tips for finding products with specific color and size under budget</query>
<query>

Re-ranking

You are about to attempt a shopping task in the WebShop environment.
Task: {TASK}
Initial Observation: {INITIAL_OBSERVATION}
Below are {K} past experiences retrieved from memory. Each is labeled with an ID.
{CANDIDATE_EXPERIENCES}
Rank these experiences from MOST useful to LEAST useful for the current task. Consider which experience addresses the specific challenges you expect to face.
Output ONLY the ranked IDs as a comma-separated list within <rank> </rank> tags.

Utilization

You are an expert autonomous agent operating in the WebShop e-commerce environment.
[Injected if a skill is selected:]
Past reflections on similar tasks: {SKILL.strat}
Warning: These lessons may be outdated. Use them only if they align with your current situation.
Your task is to: {TASK}.
Prior to this step, you have already taken {N} step(s). Below are the most recent {W} observations and the corresponding actions you took: {ACTION_HISTORY}
You are now at step {CURRENT_STEP} and your current observation is: {OBSERVATION}.
Your admissible actions: [{AVAILABLE_ACTIONS}].
You should first reason step-by-step within <think> </think> tags, then choose an admissible action within <action> </action> tags.

Distillation

You are an expert evaluating a WebShop shopping attempt.
Your task is to: {TASK}
The task was {successfully/unsuccessfully} completed.
Trajectory of the attempt: {TRAJECTORY}
<think> Analyze: What subtasks were attempted (search, filter, select, purchase)? Which succeeded or failed? What specific actions led to this outcome? What are the most valuable lessons? </think>
Output your evaluation as JSON:
{
 "task_success": ...,
 "action_lesson": "...",
 "navigation_lesson": "...",
 "description_head": "[WHEN this lesson is useful -- general task type, not specific task]"
}

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction clearly state our contributions and scope.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: We discuss limitations in §5.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper does not include theoretical proofs; the contribution is empirical.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Full hyperparameters are in §4.1 and Appendix C. Code is included in the supplemental material.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code is included in the abstract section.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Training and test details are in §4.1 and Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Main results are averaged over three random seeds. Statistical analysis is in Appendix D.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We report our computation resources in (§4.1). Complexity discussion is in §4.3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.

- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: This research conforms with the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We discuss both positive and negative societal impacts in Appendix E.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper poses no such risks. We do not release pretrained language models or scraped datasets.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cite all datasets and models used. Qwen2.5 is under Apache 2.0.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: This paper does not release new datasets or pretrained models.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper does not involve research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [No]

Justification: LLMs are used only for writing assistance purposes and do not impact the core methodology or scientific rigor of this research.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.